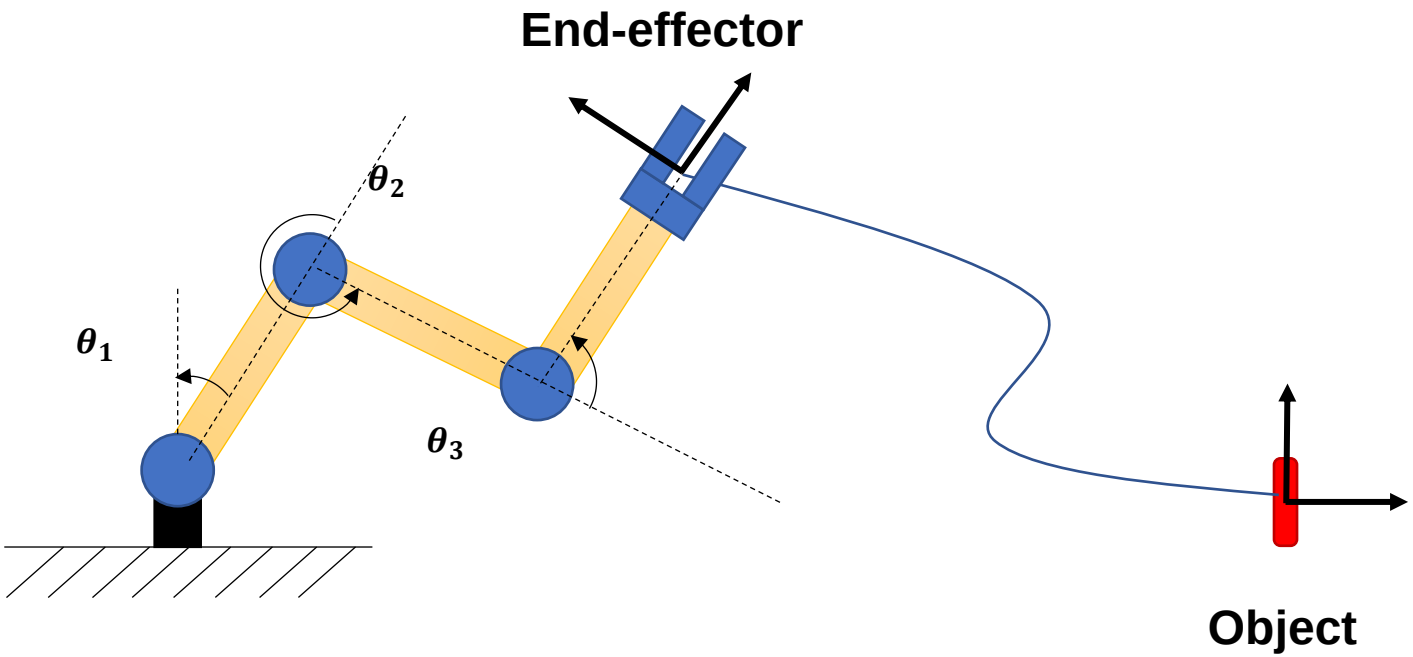
Differential Kinematics: Velocities, Accelerations and Jacobians

Topics

- Velocities in Cartesian Space
- Accelerations in Cartesian Space
- Jacobian matrices

Complementary Reading: J.J. Craig, Introduction to Robotics, Chapter 5,9.

Motivating Example



High-level goal: move robot to grasp the object

- Trajectory could be straight line
- Trajectory could be curved line (avoid obstacles, satisfy constraints, etc.)
- In both situations, often better to define the trajectory in Cartesian Space
  - How then do we determine $\{\dot{p}(t), \dot{R}(t)\} \rightarrow \dot{\theta}(t)$?

First, need to better define position and angular velocities.

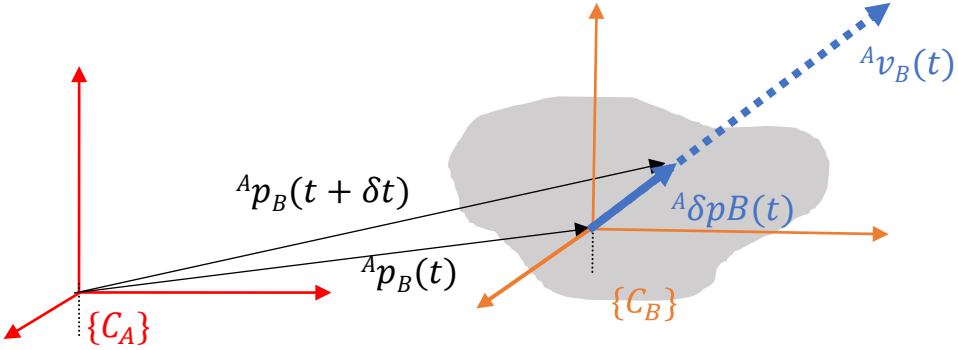
Translational and Rotational Velocities

Velocities are vectors, which means they are expressed w.r.t. coordinate frames.

- Two types of velocities, **rotational** *and* **translational**.

Translation Velocities

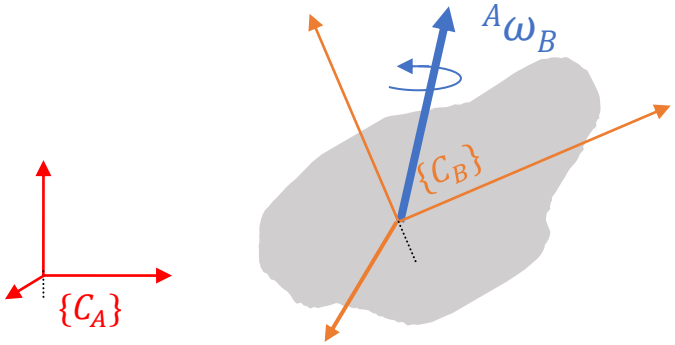
- Describe how objects/frames move *without rotation* in space.



$$ {}^{A}v_B = \frac{d}{dt}{}^{A}p_B $$

Rotational Velocities

- The angular velocity, or change in the orientation, $\dot{R}$, has a special form.
  - Cannot simply take $\frac{d}{dt}$ of the elements of R



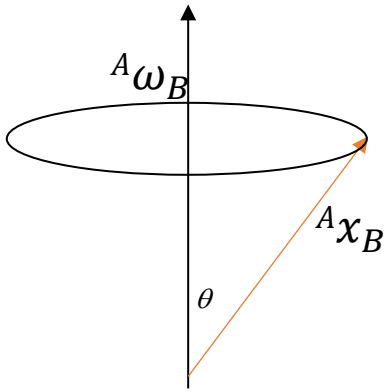
- Frame B has an orientation ^A_BR and its rotational motion may be represented by the rotational (angular) velocity vector $^A\omega_B \in \mathbb{R}^3$
  - $\|^A\omega_B\|$ defines speed of rotation
  - $^A\omega_B/\|^A\omega_B\|$: defines axis of rotation (defined in $\{A\}$)

Definitions of angular velocity with $\dot{R}$

- The angular velocity described in orientation is defined by the **change in basis vectors**

$$\dot{R} = \left[{}^{A}\dot{u}_x, {}^{A}\dot{u}_y, {}^{A}\dot{u}_z\right]$$

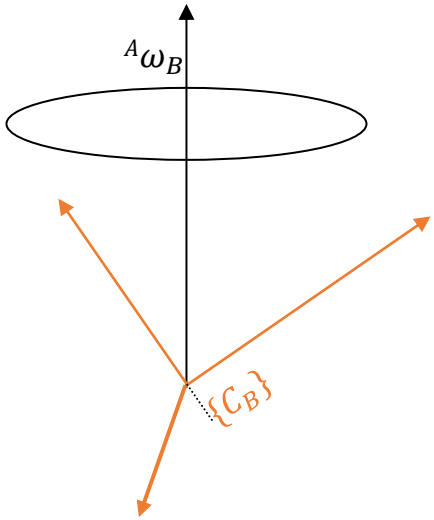
- Consider coordinate $\{C_B\}$ and its basis vector about x. If it processes about ${}^{A}\omega_B$, then the change in ${}^{A}u_{x,b}$ has…
  - Magnitude: $\|{}^{A}\omega_B\|\left\|{}^{A}u_{x,B}\right\| \sin\theta = \|{}^{A}\omega_B\| \sin\theta$
  - Direction: perpendicular to plane spanned by ${}^{A}\omega_B, {}^{A}u_{x,B}$



- Thus, it is exactly the cross product

$${}^{A}\dot{x}_B = {}^{A}\omega_B \times {}^{A}u_{x,B}$$

- Therefore, for each basis vector,



$$
\begin{aligned}
{}^A\dot{u}_{x,B} &= {}^A\omega_B \times {}^Au_{x,B} = {}^A\hat{\omega}_B {}^Au_{x,B} \\
{}^A\dot{u}_{y,B} &= {}^A\omega_B \times {}^Au_{y,B} = {}^A\hat{\omega}_B {}^Au_{y,B} \\
{}^A\dot{u}_{z,B} &= {}^A\omega_B \times {}^Au_{z,B} = {}^A\hat{\omega}_B {}^Au_{z,B}
\end{aligned}
$$

Can be written in matrix form, i.e. $\dot{R}$

Let $\hat{\omega}$ be

$$
\hat{\omega} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}
$$

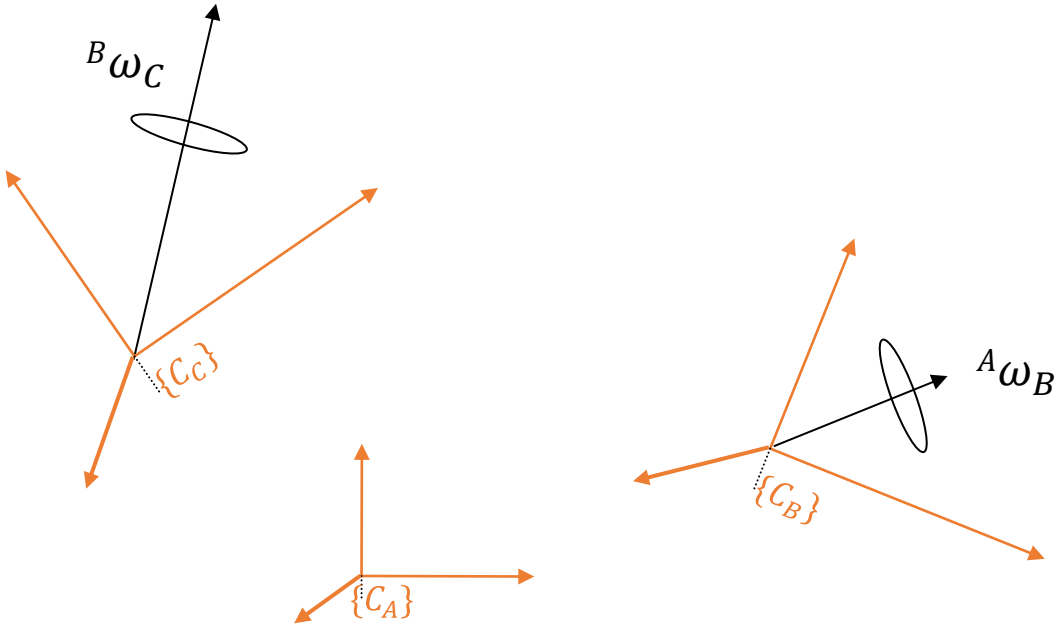
Given $\dot{R} = \left[{}^A\dot{u}_x, {}^A\dot{u}_y, {}^A\dot{u}_z\right]$ and ${}^A_BR = [{}^Au_{x,B}, {}^Au_{y,B}, {}^Au_{z,B}]$, Then

$$
{}^A\dot{R}_B = {}^A\hat{\omega}_B {}^A_BR
$$

Describing Angular Velocities in different Frames

- Describing angular velocities in different coordinate frames is quite straightforward:

$$^{A}\omega_{C} = {}^{A}\omega_{B} + {}^{A}_{B}R\,{}^{B}\omega_{C}$$



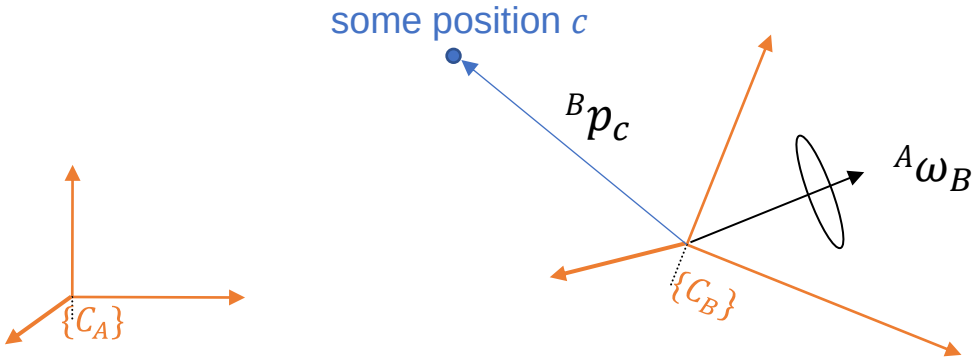
- Describing angular velocities using $\omega \in \mathbb{R}^3$ is useful for control.
- Assume $\{C_B\}$ and $\{C_A\}$ are rigidly attached to one body. What is the relation between $^{\mathrm{A}}\omega_{\mathrm{B}}$ and $^{\mathrm{B}}\omega_{c}$?

Describing Translational Velocity in Different Frames

- More complicated since rotating coordinate frames play a role.
- Find by differentiating position:

$$ {}^{A}p_C = {}^{A}p_B + {}^{A}_{B}R\ {}^{B}p_C $$

$$ \begin{aligned} {}^{A}v_C = {}^{A}\dot{p}_C &= {}^{A}\dot{p}_B + \frac{d}{dt}\left({}^{A}_{B}R\ {}^{B}p_C\right) \\ &= {}^{A}v_B + {}^{A}_{B}R\,{}^{B}\dot{p}_C + {}^{A}_{B}\dot{R}\ {}^{B}p_C \\ &= {}^{A}v_B + {}^{A}_{B}R\,{}^{B}v_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\ {}^{B}p_C \end{aligned} $$



Translational and Angular Accelerations

For equations of acceleration, simply differentiate the former velocity equations.

Rotational Acceleration:

$$
\begin{aligned}
{}^{A}\omega_C &= {}^{A}\omega_B + {}^{A}_{B}R\,{}^{B}\omega_C \\
{}^{A}\alpha_C = {}^{A}\dot{\omega}_C &= {}^{A}\dot{\omega}_B + \frac{d}{dt}\left({}^{A}_{B}R\,{}^{B}\omega_C\right) \\
&= {}^{A}\alpha_B + {}^{A}_{B}\dot{R}\,{}^{B}\omega_C + {}^{A}_{B}R\,{}^{B}\dot{\omega}_C \\
&= {}^{A}\alpha_B + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}\omega_C + {}^{A}_{B}R\,{}^{B}\alpha_C
\end{aligned}
$$

Translational Acceleration:

$$
{}^{A}v_C = {}^{A}v_B + {}^{A}_{B}R\,{}^{B}v_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}p_C
$$

$$
\begin{aligned}
{}^{A}a_C = {}^{A}\dot{v}_C &= {}^{A}\dot{v}_B + {}^{A}_{B}\dot{R}\,{}^{B}v_C + {}^{A}_{B}R\,{}^{B}\dot{v}_C + {}^{A}\dot{\hat{\omega}}_B\,{}^{A}_{B}R\,{}^{B}p_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}\dot{R}\,{}^{B}p_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}\dot{p}_C \\
&= {}^{A}a_B + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}v_C + {}^{A}_{B}R\,{}^{B}a_C + {}^{A}\dot{\hat{\omega}}_B\,{}^{A}_{B}R\,{}^{B}p_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}\dot{R}\,{}^{B}p_C + {}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}v_C \\
&= {}^{A}a_B + 2\,{}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}v_C + {}^{A}_{B}R\,{}^{B}a_C + {}^{A}\hat{\alpha}_B\,{}^{A}_{B}R\,{}^{B}p_C + {}^{A}\hat{\omega}_B\,{}^{A}\hat{\omega}_B\,{}^{A}_{B}R\,{}^{B}p_C
\end{aligned}
$$

Notes:

- Consider always apply the general form
- *Very easy* to make a mistake if you forgot some terms if doing differentiation on a robot kinematics
  - e.g. coordinate frame orientations line up and ${}^A_B R = I$ and you omit R during differentiation
  - e.g. coordinate frame origins line up and ${}^{\mathrm{A}}p_B = 0$ and you omit ${}^{\mathrm{A}}p_B$ during differentiation

The Jacobian Matrix

- Now we are ready to describe the relationship between the joint velocities and the end effector velocities (both translational and angular), using the *Jacobian matrix.*

Deriving the Jacobian Matrix from Forward Kinematics:

$$[x, y, z, \omega_x, \omega_y, \omega_z] \stackrel{\text{def}}{=} \chi = f(\theta) \stackrel{\text{def}}{=} \begin{bmatrix} f_1(\theta_1 \dots \theta_n) \\ \vdots \\ f_6(\theta_1 \dots \theta_n) \end{bmatrix}$$

Expand using Taylor series:

$$x = f_1(\theta_0 + \delta\theta) \approx f_1(\theta_0) + \left[\frac{\partial f_1(\theta_0)}{\partial \theta_1} \dots \frac{\partial f_1(\theta_0)}{\partial \theta_n}\right] [\delta\theta_1 \dots \delta\theta_n]^\top + O(\theta^2)$$

$$f_1(\theta_0 + \delta\theta) - f_1(\theta_0) = \left[\frac{\partial f_1(\theta_0)}{\partial \theta_1} \dots \frac{\partial f_1(\theta_0)}{\partial \theta_n}\right] [\delta\theta_1 \dots \delta\theta_n]^\top + O(\theta^2)$$

$$\dot{x}|_{\theta_0} \approx \left[\frac{\partial f_1(\theta_0)}{\partial \theta_1} \dots \frac{\partial f_1(\theta_0)}{\partial \theta_n}\right] \dot{\theta} + O(\theta^2)$$

$$\approx J_1(\theta_0)\dot{\theta}$$

- Assume that we have an n-link robot with joint variables $\theta = \theta_1, \theta_2, \ldots, \theta_N$

$$^0\chi_N = {}^0\left[x, y, z, \omega_x, \omega_y, \omega_z\right]_N^{\mathsf{T}} = f(\theta)$$

- If we differentiate $^0\chi_N = f(\theta)$ over time,

$$^0\dot{\chi}_N = {}^0\left[\dot{x}, \dot{y}, \dot{z}, \dot{\omega}_x, \dot{\omega}_y, \dot{\omega}_z\right]_N^{\mathsf{T}} = \dot{f}(\theta)$$

and

$$^0\dot{\chi}_N = {}^0\left[\dot{x}, \dot{y}, \dot{z}, \dot{\omega}_x, \dot{\omega}_y, \dot{\omega}_z\right]_N^{\mathsf{T}} \approx J(\theta)\dot{\theta}$$

where

$$J(\theta) = \begin{bmatrix} \frac{\partial f_1(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_1(\theta)}{\partial \theta_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_6(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_6(\theta)}{\partial \theta_N} \end{bmatrix}$$ is the Jacobian matrix

What does this Jacobian mapping mean?

$$J(\theta) = \begin{bmatrix} \frac{\partial f_1(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_1(\theta)}{\partial \theta_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_6(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_6(\theta)}{\partial \theta_N} \end{bmatrix}$$

- **Each row describes** the *effect* of *every joint's velocities* on one axis of velocity ($\dot{x}, \dot{y}, \dot{z}, \dot{\omega}_x, \dot{\omega}_y, \dot{\omega}_z$) of the end effector
- **Each column describes** the *effect* of *one* joint's velocities on every axis of velocity ($\dot{x}, \dot{y}, \dot{z}, \dot{\omega}_x, \dot{\omega}_y, \dot{\omega}_z$) of the end effector

Most importantly, through the *Jacobian matrix inverse* J^{-1}, one can determine how to the joint angles should change to produce a change in output pose:

$$\begin{bmatrix} \dot{\theta}_1 \\ \vdots \\ \dot{\theta}_N \end{bmatrix} = J^{-1}(\theta) \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\omega}_x \\ \dot{\omega}_y \\ \dot{\omega}_z \end{bmatrix}$$

Why is it $\dot{r}$, $\dot{p}$, $\dot{z}$ and not ω?

- orientations described in roll r, pitch p, and yaw y are describing a coordinate frame with orientation

$$R(r,p,y) = R_x(r)R_y(p)R_z(y)$$

- The conversion can be found by taking $\frac{d}{dt}R(r,p,y)$ and pulling out terms $\dot{r}, \dot{p}$, and $\dot{z}$ such that

$$\omega = B(r,p,y)\begin{bmatrix}\dot{r}\\ \dot{p}\\ \dot{y}\end{bmatrix} = \begin{bmatrix}1 & 0 & \sin p\\ 0 & \cos r & -\cos p \sin r\\ 0 & \sin r & \cos p \cos r\end{bmatrix}\begin{bmatrix}\dot{r}\\ \dot{p}\\ \dot{y}\end{bmatrix}$$

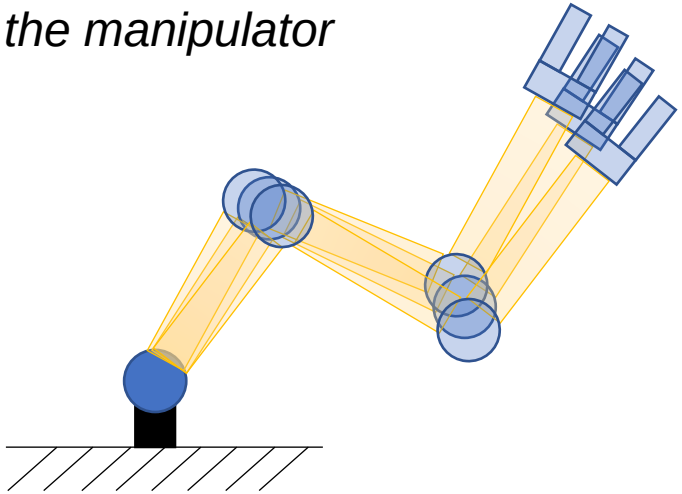
- Can add this relationship the Jacobian equations to get ω out:

$${}^0[\dot{p}, \omega]_N^{\mathsf{T}} \approx \begin{bmatrix}I_{3\times 3} & 0\\ 0 & B\end{bmatrix} J(\theta)\dot{\theta}$$

Using the Jacobian Matrix for Control

The Jacobian matrix is a *linear approximation of the manipulator*

- i.e. valid in a close proximity $\theta \pm \delta$.



- Linear control can be used:

  Forward Kinematics:

  $$x = f(\theta)$$

  $$x + \pm\delta x = f(\theta + \delta\theta)$$

  Approximation used for linear control:

  $$x \pm \delta x \approx f(\theta) + J(\theta)\delta\theta$$

  $$\delta x \approx J(\theta)\delta\theta$$

  Given a desired incremental desired δx, apply

  $$\delta\theta = J(\theta)^{-1}\delta x$$

Key insights

$$\begin{bmatrix} {}^0\dot{p}_N \\ {}^0\omega_N \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_1(\theta)}{\partial \theta_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_6(\theta)}{\partial \theta_1} & \cdots & \frac{\partial f_6(\theta)}{\partial \theta_N} \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \vdots \\ \dot{\theta}_N \end{bmatrix}$$

- This linear controller $\delta\theta = J(\theta)^{-1}\delta x$ involves solving the matrix inverse problem
  - If $n > 6$, fat matrix J, then you have infinite solutions and it is an underconstrained problem.
  - If $n < 6$, skinny matrix J, then you have approximate solutions and it is an overconstrained problem.
  - If $n = 6$, then you have a countable number of solutions and it is an fully constrained problem.

(the above assumes full rank J)

Pseudoinverse of Matrices

Pseudoinverse is used in practice for matrix inverses (not only for Jacobians), to avoid rank-deficient matrix inversion.

- **Standard pseudo-inverse** (not advised)

$$J^{\dagger} = (J^{\top}J)^{-1}J^{\top} \qquad \text{or} \qquad J^{\dagger} = J^{\top}(JJ^{\top})^{-1}$$

  - Depends on fat or thin J

- **Robust pseudo-inverse method 1: Damped Least Squares**

$$J^{\dagger} = (J^{\top}J + \lambda^2 I)^{-1}J^{\top} \qquad \text{or} \qquad J^{\dagger} = J^{\top}(JJ^{\top} + \lambda^2 I)^{-1}$$

- **Robust pseudo inverse method 2: Singular Value Decomposition**

$$J = U\Sigma V^{\top} \;\rightarrow\; J^{\dagger} = V\Sigma^{\dagger}U^{\top}$$

  - where the diagonal on $\Sigma^{\dagger}$are $\mathrm{diag}(\sigma_{max}^{-1}, \dots, \sigma_i^{-1}, \dots, \sigma_{min}^{-1}, 0 \dots)$ and may include trailing zeros
  - where if $\frac{\sigma_i}{\sigma_{max}} < \mathrm{min_ratio}$ then $\sigma_{i \dots min}^{-1} = 0.$

4. A Jacobian can be found for *any* kinematically-defined parameters given inputs

- Intermediate Joint Features (joint poses, CoM poses, etc.)
- Metrics involving kinematics (e.g. inverse kinematic L2 error)

Algorithm Method of Gauss-Newton

input : $f : \mathbb{R}^n \to \mathbb{R}$ a function such that $f(\mathbf{x}) = \sum_{i=1}^{m} (f_i(\mathbf{x}))^2$
where all the f_i are differentiable functions from $\mathbb{R}^n$ to $\mathbb{R}$
$\mathbf{x}^{(0)}$ an initial solution

output: $\mathbf{x}^\star$, a local minimum of the cost function f.

begin
  $k \leftarrow 0$;
  while STOP-CRIT **and** $(k < k_{max})$ **do**
    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \boldsymbol{\delta}^{(k)}$;
    with $\boldsymbol{\delta}^{(k)} = \arg\min_{\boldsymbol{\delta}} \|\mathcal{F}(\mathbf{x}^{(k)}) + \mathbf{J}_{\mathcal{F}}(\mathbf{x}^{(k)})\boldsymbol{\delta}\|^2$;
    $k \leftarrow k + 1$;
  return $\mathbf{x}^{(k)}$
end

Algorithm 5: Levenberg-Marquardt algorithm

input : $f : \mathbb{R}^n \to \mathbb{R}$ a function such that $f(\mathbf{x}) = \sum_{i=1}^{m} (f_i(\mathbf{x}))^2$
where all the f_i are differentiable functions from $\mathbb{R}^n$ to $\mathbb{R}$
$\mathbf{x}^{(0)}$ an initial solution

output: $\mathbf{x}^\star$, a local minimum of the cost function f.

begin
  $k \leftarrow 0$;
  $\lambda \leftarrow \max \text{diag}(\mathbf{J}^\mathsf{T}\mathbf{J})$;
  $\mathbf{x} \leftarrow \mathbf{x}^{(0)}$;
  while STOP-CRIT **and** $(k < k_{max})$ **do**
    Find $\boldsymbol{\delta}$ such that $(\mathbf{J}^\mathsf{T}\mathbf{J} + \lambda \text{diag}(\mathbf{J}^\mathsf{T}\mathbf{J}))\boldsymbol{\delta} = \mathbf{J}^\mathsf{T}\mathbf{f}$;
    $\mathbf{x}' \leftarrow \mathbf{x} + \boldsymbol{\delta}$;
    if $f(\mathbf{x}') < f(\mathbf{x})$ **then**
      $\mathbf{x} \leftarrow \mathbf{x}'$;
      $\lambda \leftarrow \frac{\lambda}{\nu}$;
    else
      $\lambda \leftarrow \nu\lambda$;
    $k \leftarrow k + 1$;
  return $\mathbf{x}$
end